

Analyzing the Role of the AI Assisted Programming in Modern Software Development

A Task Management System Based Comparative Analysis

Module : Advanced Programming Techniques – IT 3003

Assessment: AI-Assisted Programming

Name: K.G.C.Chamodya

Index: s17221

Submission Date: 2026/07/12

Table of Content

1. Introduction.....	04
2. Conceptual understanding of AI assisted programming (Task 01)	
2.1 AI Assisted Programming.....	05
2.2 Types of AI Coding Tools.....	05
2.3 Advantages in Software Development.....	05
2.4 Risks and Limitations.....	06
3. Practical Implementation (Task 02)	
3.1 Problem Selected.....	07
3.2 System Design and UML Diagrams.....	07
3.3 Version A (Traditional Implementation).....	10
3.4 Version B (AI Assisted Implementation).....	12
3.5 AI Prompts used in Implementing Version B.....	14
3.6 Sample Program Execution.....	16
3.7 Comparison between Version A and Version B.....	18
4. Critical Analysis (Task 03)	
4.1 How AI Changed the Coding Approach.....	20
4.2 Did AI Improved or Reduced Understanding?	20
4.3 When AI Should Not Be Used?.....	20
4.4 Ethical Considerations in AI Generated Code.....	21
5. Conclusion (Task 04).....	22
6. GitHub Repository.....	23

Table of Figures

Figure 1 : use case Diagram for the Task Management System.....	07
Figure 2 : Class Diagram for the Task Management System.....	08
Figure 3: General Activity Diagram.....	08
Figure 4: Activity Diagram for “Add Task to Project Option”.....	09
Figure 5: Sequence Diagram.....	10
Figure 6: Some code sections from main.java in Version A.....	11
Figure 7: Some code sections from projectManager.java in Version A.....	12
Figure 8: TaskManagerException.java class.....	13
Figure 9: Validation inside the Task.java constructor.....	13
Figure 10: Try-catch code inside the main.java.....	14
Figure 11: Prompts used to Implement Version B.....	15
Figure 12: Output of version A.....	16
Figure 13: Invalid input for version A.....	18
Figure 14: Invalid input for version B.....	18

1. Introduction

With the use of technologies like AI code generators. Debugging assistants, and smart programming support systems, artificial intelligence is becoming more and more incorporated into modern software production. These technologies are transforming the way developers create, construct, test and manage software applications. A portion of the coding process is now collaborative between developer and an AI assistant rather than entirely manual.

By creating the same application, a console based task management system, two times, once using a strictly conventional, manually written approach (Version A) and another one using an AI assisted approach (Version B) this report investigate the role of AI assisted programming in software engineering. Development time, code quality, error handling, readability and maintainability are all evaluated between these two systems. The report concludes with a critical analysis of the ethical issues surrounding AI aid, including when it can be beneficial and when it might be harmful.

2. Conceptual understanding of AI assisted programming (Task 01)

2.1 AI Assisted Programming

The use of artificial intelligence (AI) tools, usually based on massive language models, to assist a developer in writing, analyzing debugging or improving software is known as “AI assisted Programming.” These tools function as an aid integrated into the development process rather than taking the place of the developer. They can suggest code as it entered, generate entire functions or classes from a natural language description, explained error messages or recommend refactors. Whatever the AI generates must still be reviewed, tested and integrated by the programmer.

2.2 Types of AI Coding Tools

- Code generation technologies, such as GitHub copilot, Amazon Q Developer and Tabnine can produce or recommended code based on comments or natural language instructions.
- Conversational Assistants are chat based programs like ChatGpt and Claude that, given a problem description, can create a function, explain a notion or construct a class.
- Debugging Assistants are programs that analyze a track trace or a test that fails and recommended a likely reason and a solution.
- Refactoring and code review tools recommend structural enhancements such removing redundant logic, renaming parameters or extracting methods.
- Test generation tools creates unit tests from pre-existing code to increase coverage while requiring less manual labor.
- Documentation generators create README files, API documentation from existing codes.

2.3 Advantages in Software Development.

- Standard and repetitive codes (getters/setters, CRUD operations,..) can be produce more quickly.

- Reduce the difficulty of utilizing new language features or APIs which promotes learning.
- Initial recommendations for addressing errors and edge scenarios that a developer could overlook.
- Quicker creation of supporting artifacts, including tests, documents and standard sample data.
- The capacity to swiftly investigate different designs (for instance, contrasting enums with raw strings for a specific set of values).

2.4 Risks and Limitations

- Artificially generated code : The AI may reliably produce code that calls unknown methods or libraries or that appears proper but is slightly incorrect.
- Dependency and skill loss : Students or less experienced developers may take advice without fully understanding it, which impairs their capacity to solve problems on their own.
- Security concerns: If AI generated code is not thoroughly examined, it may contain insecure patterns (such as missing input validations).
- Limited project context: AI tools might not completely analyze the larger codebase, resulting in recommendations that makes sense locally but are incorrect globally.
- Concerns about originality and licensing: generated code from the model's training data could copy already existing copyrighted code.
- False Confidence: Without independent verification, it is simple to place too much trust in the result due to its fluency and proper formatting.

3. Practical Implementation (Task 02)

3.1 Problem Selected

The chosen project is a task management system, which is a console based java program that allows users to register individuals (users), arrange tasks into projects, add new tasks to projects, and assign tasks to users, update the status of tasks and see the project data. This problem was selected due to its moderate complexity, which includes multiple connected classes, one to many relationships and sufficient business criteria (valid priority, valid statuses, and valid dates) to clearly distinguish between traditional and AI assisted approach.

The identical problem was implemented in two separate ways,

- Version A (Traditional Method) : coded by hand using developer's own understanding of fundamental of Java, without the aid of AI code generation tools.
- Version B (AI Assisted Method) : Created with the aid of AI coding assistants that was asked to supplement the same fundamental concept with additional features, structure and validation.

3.2 System Design and UML Diagrams

The System's general design is depicted in the following diagrams. While version A implements a subset of identical use cases without search, track overdue tasks/projects or file persistence and without the validation depicted in the class diagram. Following diagrams are drawn for the Version A (traditional method).

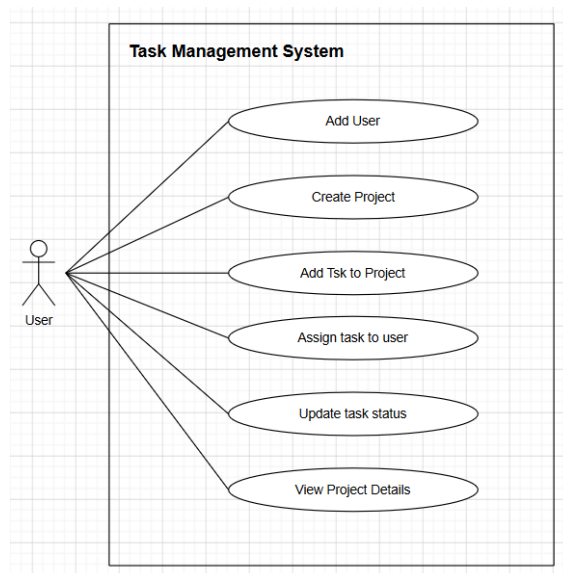


Figure 1 : use case Diagram for the Task Management System

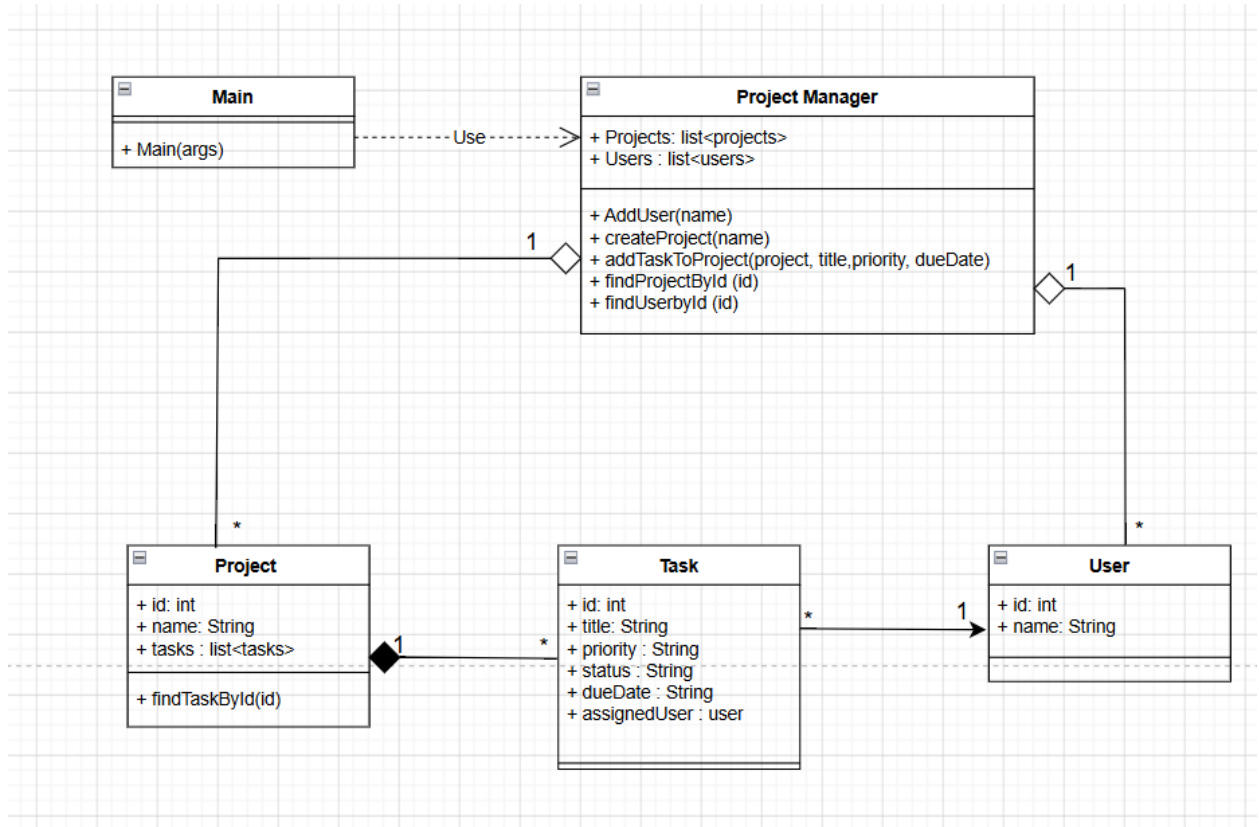


Figure 2 : Class Diagram for the Task Management System.

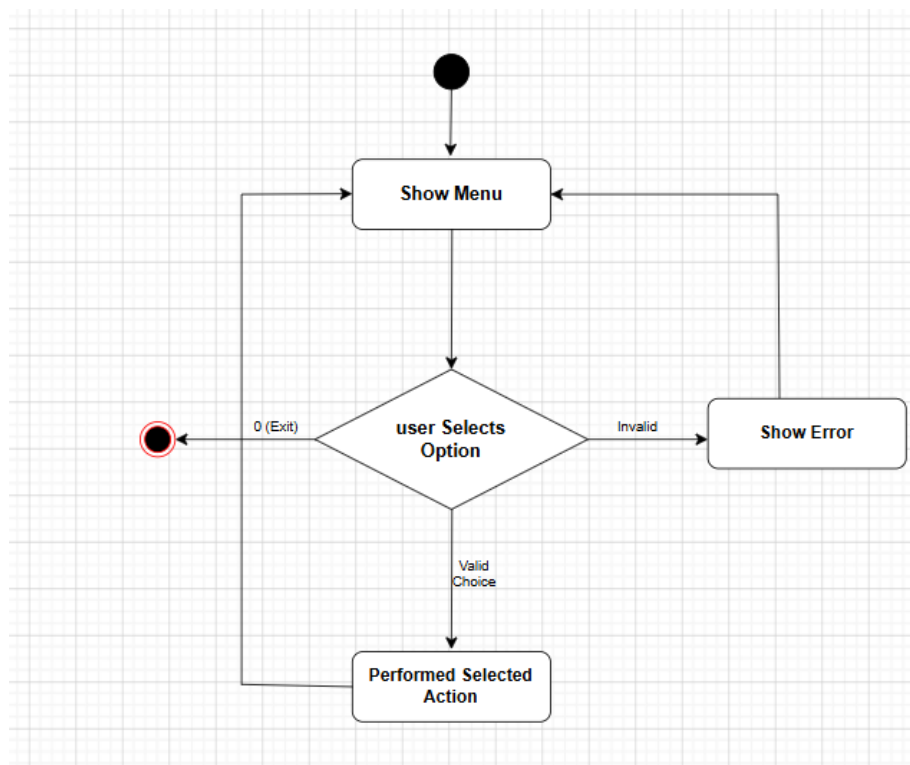


Figure 3: General Activity Diagram.

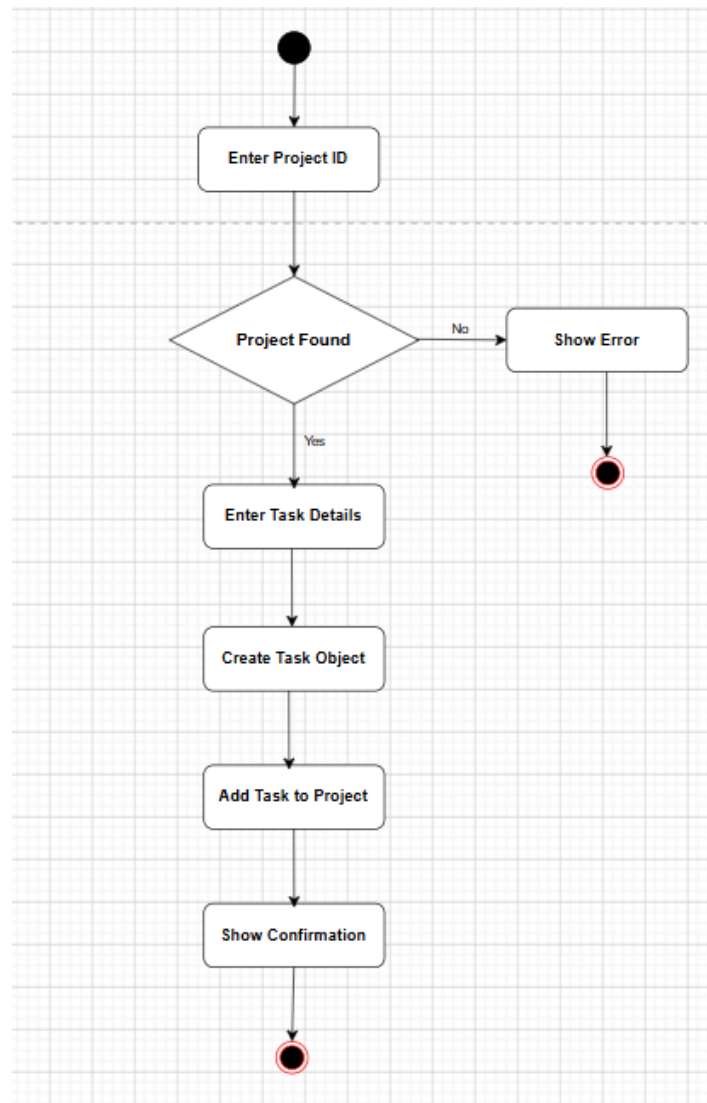


Figure 4: Activity Diagram for “Add Task to Project Option”

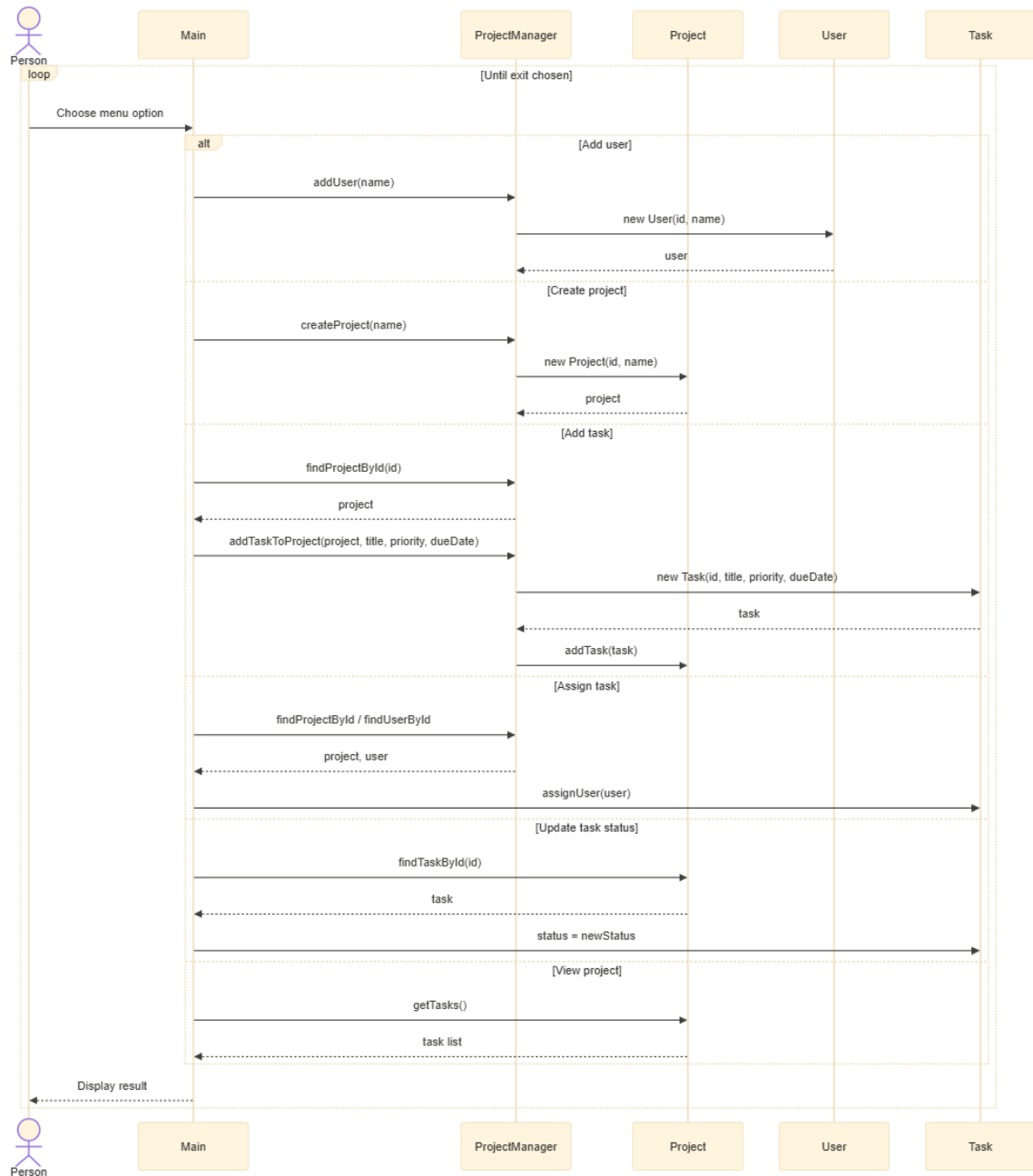


Figure 5: Sequence Diagram

3.3 Version A (Traditional Implementation)

All the writing in version A was done by hand. The due date, priority and the status are kept in plain the program depends on the user consistently supplying well formed data, String fields, input validation and exception handling are all absent.

```

1  import java.util.Scanner;
2
3  public class Main { @chalaniChamodya
4      static ProjectManager manager = new ProjectManager(); no usages
5      static Scanner scanner = new Scanner(System.in); no usages
6
7      public static void main(String[] args) { @chalaniChamodya
8          boolean running = true;
9          while (running) {
10             printMenu();
11             System.out.print("Choose an Option: ");
12             int choice = Integer.parseInt(scanner.nextLine());
13             if (choice == 1) {
14                 createUser();
15             } else if (choice == 2) {
16                 createProject();
17             } else if (choice == 3) {
18                 addTask();
19             } else if (choice == 4) {
20                 assignTask();
21             } else if (choice == 5) {
22                 updateTaskStatus();
23             } else if (choice == 6) {
24                 viewProject();
25             } else if (choice == 0) {
26                 running = false;
27             } else {
28                 System.out.println("Invalid option, try again.");
29             }
30         }
31
32     }
33
34     static void printMenu() { no usages @chalaniChamodya
35         System.out.println("\n=== Task Manager ===");
36         System.out.println("1. Add User");
37         System.out.println("2. Create Project");
38         System.out.println("3. Add Task to Project");
39         System.out.println("4. Assign Task to User");
40         System.out.println("5. Update Task Status");
41         System.out.println("6. View Project Details");
42         System.out.println("0. Exit");
43     }
44
45     static void createUser() { no usages @chalaniChamodya
46         System.out.print("Enter user name: ");
47         String name = scanner.nextLine();
48         User user = manager.addUser(name);
49         System.out.println("Created: " + user);
50     }
51
52     static void createProject() { no usages @chalaniChamodya
53         System.out.print("Enter project name: ");
54         String name = scanner.nextLine();
55         Project project = manager.createProject(name);
56         System.out.println("Created: " + project);
57     }

```

Figure 6: Some code sections from main.java in Version A.

```

1  import java.util.ArrayList;
2  import java.util.List;
3  |
4  // Keeps track of all projects and users in the system
5  public class ProjectManager { no usages  @chalaniChamodya
6      List<Project> projects = new ArrayList<>(); no usages
7      List<User> users = new ArrayList<>(); no usages
8      int nextProjectId = 1; no usages
9      int nextTaskId = 1; no usages
10     int nextUserId = 1; no usages
11
12     public User addUser(String name) { no usages  @chalaniChamodya
13         User user = new User(nextUserId++, name);
14         users.add(user);
15         return user;
16     }
17
18     public Project createProject(String name) { no usages  @chalaniChamodya
19         Project project = new Project(nextProjectId++, name);
20         projects.add(project);
21         return project;
22     }
23
24     public Task addTaskToProject(Project project, String title, String priority, String dueDate) {
25         Task task = new Task(nextTaskId++, title, priority, dueDate);
26         project.tasks.add(task);
27         return task;
28     }
29
30     public Project findProjectById(int id) { no usages  @chalaniChamodya
31         for (Project p : projects) {
32             if (p.id == id) {
33                 return p;
34             }
35         }
36         return null;
37     }
38
39     public User findUserById(int id) { no usages  @chalaniChamodya
40         for (User u : users) {
41             if (u.id == id) {
42                 return u;
43             }
44         }
45         return null;
46     }
47 }
48

```

Figure 7: Some code sections from projectManager.java in Version A.

3.4 Version B (AI Assisted Implementation)

Version B was expanded and strengthened with AI support, although it maintains the same general concept (users, projects, tasks handle by the project manager). Enums for task status and priority rather than raw Strings, a custom checked exception (TaskManagerException) for application level error, constructor level validation, correct LocalDate parsing for due dates and a keyword search and overdue task feature are just a few of the ideas that the AI proposed and assisted in implementing. Additionally, all public states were converted to private fields with getters and setters.

```

1  /**
2   * Thrown when a task or project operation receives invalid input
3   * (e.g. an unknown ID, a bad date format, an empty title).
4   * Having a dedicated exception type is one of the things AI assistance
5   * suggested over Version A's approach of just letting the program crash.
6   */
7  public class TaskManagerException extends Exception {
8      >    public TaskManagerException(String message) { super(message); }
11 }
12

```

Figure 8: TaskManagerException.java class.

```

10 public class Task implements Comparable<Task>, Serializable {
11     private static final long serialVersionUID = 1L;
12
13     private final int id;
14     private String title;
15     private Status status;
16     private Priority priority;
17     private LocalDate dueDate;
18     private User assignedUser;
19
20     public Task(int id, String title, Priority priority, String dueDate) throws TaskManagerException {
21         if (title == null || title.trim().isEmpty()) {
22             throw new TaskManagerException("Task title cannot be empty.");
23         }
24         if (priority == null) {
25             throw new TaskManagerException("Task priority cannot be null.");
26         }
27         this.id = id;
28         this.title = title.trim();
29         this.priority = priority;
30         this.dueDate = parseDate(dueDate);
31         this.status = Status.TODO;
32     }
33
34     private LocalDate parseDate(String dateStr) throws TaskManagerException {
35         try {
36             return LocalDate.parse(dateStr.trim());
37         } catch (DateTimeParseException | NullPointerException e) {
38             throw new TaskManagerException("Invalid due date '" + dateStr + "'. Use format YYYY-MM-DD.");
39         }
40     }
41 }

```

Figure 9: Validation inside the Task.java constructor.

```

16     public static void main(String[] args) { @chalaniChamodya
17         boolean running = true;
18         while (running) {
19             printMenu();
20             int choice = readInt("Choose an option: ");
21             try {
22                 switch (choice) {
23                     case 1: createUser(); break;
24                     case 2: createProject(); break;
25                     case 3: addTask(); break;
26                     case 4: assignTask(); break;
27                     case 5: updateTaskStatus(); break;
28                     case 6: viewProject(); break;
29                     case 7: searchTasks(); break;
30                     case 8: viewOverdueTasks(); break;
31                     case 9: saveData(); break;
32                     case 10: loadData(); break;
33                     case 0: running = false; break;
34                     default: System.out.println("Invalid option, try again.");
35                 }
36             } catch (TaskManagerException e) {
37                 // Application-level errors (bad ID, empty name, bad date) land here
38                 System.out.println("Error: " + e.getMessage());
39             } catch (IllegalArgumentException e) {
40                 // e.g. Priority.valueOf() or Status.valueOf() given an unknown word
41                 System.out.println("Error: invalid value entered - " + e.getMessage());
42             }
43         }
44         System.out.println("Goodbye!");
45     }

```

Figure 10: Try-catch code inside the main.java.

Instead of being fully reproduced here, the GitHub repository contains the entire source code for both Version A Version B (Main.java, ProjectManager.java, Task.java, User.java).

3.5 AI Prompts used in Implementing Version B

The instructions provided to the AI helper when constructing version B on top of the existing functional version A code are shown by the prompts below. (Following are the some of the prompts that I used.)

Version_A.zip

ZIP

Here is my working Task Manager (Version A). Convert the priority and status fields from String to proper Java enums, and update every place that uses them.

Architected enum conversion strategy with input validation and retry logic

Now updating `Task.java` to use the enums instead of Strings:

Edited a file, viewed a file

Good — `toString()` needs no change since string concatenation automatically calls `.toString()` on enums, which prints the constant name (e.g. `HIGH`, `IN_PROGRESS`) just like before.

Now `ProjectManager.java`:

Add a custom checked exception and use it, plus input validation, so invalid input like an empty name or a bad date format does not crash the console app.

Architected comprehensive rewrite with exception handling and input validation

Now updating `Main.java` to use these everywhere input can go wrong — the menu loop, IDs, names, and dates:

Edited 3 files

Now `addTask()` — validate the project ID, title, and date:

Make Task implement Comparable so a project's tasks can be sorted by due date, and add a method that returns only the overdue tasks.

Charted implementation strategy for comparable tasks and filtering methods

Now let's add the sorting and overdue-filtering methods to `Project.java`, since the request frames this as "a project's tasks":

Add `sortTasksByDueDate` and `getOverdueTasks` methods to Project

Now let's wire this into the console menu so it's actually usable — two new options: sort a project's tasks by due date, and view overdue tasks:

Add a case-insensitive keyword search over task titles within a project.

Viewed a file, edited a file

Now wiring this into `Main.java` as menu option 9, following the same validation pattern used elsewhere:

Edited 2 files

Now add the `searchTasks()` method itself, with input validation so an empty keyword doesn't slip through:

Add the ability to save and load the entire ProjectManager state to a file using Java serialization.

Architected serialization framework and integrated file persistence mechanisms

Let's make all the model classes `Serializable`, then add save/load methods to `ProjectManager`. First, `User.java`:

Make User implement Serializable

Now `Task.java`:

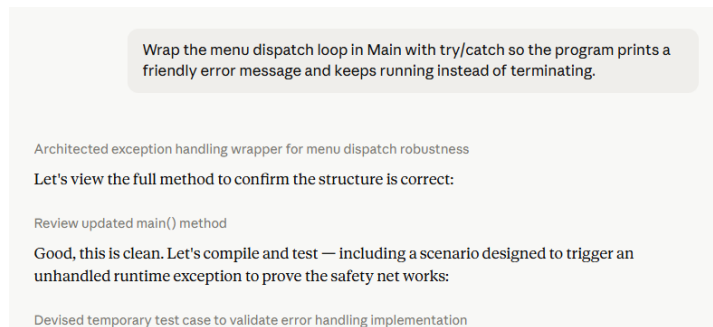


Figure 11: Prompts used to Implement Version B

Before being approved, each AI suggestion was examined line by line. For example, method names were compared to the rest of the code, the generated validation messages were reworded to fit the overall style of the program, and the AI's initial attempt at the file persistence, which initially serialized only the list of tasks, was manually corrected to serialize the entire Project Manager, preserving users and IDs throughout a save cycle as well.

3.6 Sample Program Execution

To make sure they function as described, both versions were compiled and executed from the command line. We can replicate the console sessions below by building each version's src folder with javac and executing java Main. These are real captured outputs, not mock-ups.

Version A

```
PS D:\UoC\3rd Year\1st Sem\IT 3003 - Advanced Programming Techniques\Assignment 02\Assignment\Version A - Traditional Method\src> javac *.java
PS D:\UoC\3rd Year\1st Sem\IT 3003 - Advanced Programming Techniques\Assignment 02\Assignment\Version A - Traditional Method\src> java Main

=== Task Manager ===
1. Add User
2. Create Project
3. Add Task to Project
4. Assign Task to User
5. Update Task Status
6. View Project Details
0. Exit
Choose an Option: 1
Enter user name: John Peter
Created: John Peter (ID: 1)

=== Task Manager ===
1. Add User
2. Create Project
3. Add Task to Project
4. Assign Task to User
5. Update Task Status
6. View Project Details
0. Exit
Choose an Option: 2
Enter project name: Develop a Learning Management System
Created: Project: Develop a Learning Management System (ID: 1, Tasks: 0)

=== Task Manager ===
1. Add User
2. Create Project
3. Add Task to Project
4. Assign Task to User
5. Update Task Status
6. View Project Details
0. Exit
```

```

5. Update Task Status
6. View Project Details
0. Exit
Choose an Option: 3
Enter project ID: 1
Enter task title: Design the dashboard interface.
Enter priority (LOW, MEDIUM, HIGH): LOW
Enter due date (e.g. 2026-07-15): 2026-08-30
Added: [1] Design the dashboard interface. | Priority: LOW | Status: TODO | Due: 2026-08-30 | Assigned to: Unassigned

=== Task Manager ===
1. Add User
2. Create Project
3. Add Task to Project
4. Assign Task to User
5. Update Task Status
6. View Project Details
0. Exit
Choose an Option: 4
Enter project ID: 1
Enter task ID: 1
Enter user ID: 1
Assigned John Peter to task: Design the dashboard interface.

=== Task Manager ===
1. Add User
2. Create Project
3. Add Task to Project
4. Assign Task to User
5. Update Task Status
6. View Project Details
0. Exit
Choose an Option: 5
Enter project ID: 1
Enter task ID: 1
Enter new status (TODO, IN_PROGRESS, DONE): IN_PROGRESS
Updated status: [1] Design the dashboard interface. | Priority: LOW | Status: IN_PROGRESS | Due: 2026-08-30 | Assigned to: John Peter

=== Task Manager ===
1. Add User
2. Create Project
3. Add Task to Project
4. Assign Task to User
5. Update Task Status
6. View Project Details
0. Exit
Choose an Option: 6
Enter project ID: 1
Project: Develop a Learning Management System (ID: 1, Tasks: 1)
  [1] Design the dashboard interface. | Priority: LOW | Status: IN_PROGRESS | Due: 2026-08-30 | Assigned to: John Peter

=== Task Manager ===
1. Add User
2. Create Project
3. Add Task to Project
4. Assign Task to User
5. Update Task Status
6. View Project Details
0. Exit
Choose an Option: 0
Goodbye!

PS D:\UoC\3rd Year\1st Sem\IT 3003 - Advanced Programming Techniques\Assignment 02\Assignment\Version A - Traditional Method\src>

```

Figure 12: Output of version A

Invalid Input for Version A

```
=== Task Manager ===
1. Add User
2. Create Project
3. Add Task to Project
4. Assign Task to User
5. Update Task Status
6. View Project Details
0. Exit
Choose an Option: 3
Enter project ID: First
Exception in thread "main" java.lang.NumberFormatException: For input string: "First"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:668)
    at java.base/java.lang.Integer.parseInt(Integer.java:786)
    at Main.addTask(Main.java:61)
    at Main.main(Main.java:18)
```

Figure 13: Invalid input for version A

When the project ID is entered as text rather than a number, Version A crashes.

Same Invalid Input for Version B

```
=== Task Manager (AI-Assisted Version) ===
1. Add User
2. Create Project
3. Add Task to Project
4. Assign Task to User
5. Update Task Status
6. View Project Details (sorted by due date)
7. Search Tasks by Keyword
8. View Overdue Tasks
9. Save Data to File
10. Load Data from File
0. Exit
Choose an option: 3
Enter project ID: First
Not a valid number, defaulting to -1.
Error: No project found with ID -1.

=== Task Manager (AI-Assisted Version) ===
1. Add User
2. Create Project
```

Figure 14: Invalid input for version B

Version B handles the error correctly.

3.7 Comparison between Version A and Version B

Version A – Traditional Method

- Development Time – A first working version was quickly obtained, but no time was spent on validation. Later, more time was required to trace crashes like the one in Figure 13.

- Code Quality – Public fields, bare String types for priority, status and date and no encapsulation are basic yet weak.
- Error Handling – Not one. The application ends when invalid input results in an unhandled exception. (Figure 13)
- Readability and Maintainability – A misspelling in a status string would be silently accepted despite being brief and simple to read at a glance.

Version B – AI Assisted Method

- Development time – Reviewing and modifying AI recommendations took a little longer up front, but once the pattern was found, further features (search, track overdues) were implemented fast.
- Code Quality – Stronger. Implemented comparable and serializable, priority and status enums, LocalDate for processing real dates, private fields with getters and setters.
- Error Handling – When invalid input is reported, the program continues run thanks to a specialized checked exception (TaskManagerException) and try/catch in Main (Figure 14).
- Readability and Maintainability – Overall, there is more code, but it is safer to extend because improper enum values are rejected at parse time or by the compiler rather than quietly damaging state.

4. Critical Analysis (Task 03)

4.1 How AI Changed the Coding Approach

The practice of writing every line from start was replaced by expressing an outcome and then reviewing the results when working with AI assistance. The task became, determine what should be enforced (for example, “a task must have a non-empty title”), request it, and then verify that the generated code actually enforced it at the appropriate location, rather than typing out an enum or a try/catch block from memory. This shifted focus from java grammar itself to design choices, such as what would be validated, what should be persisted and what should be an enum.

4.2 Did AI Improved or Reduced Understanding?

Seeing a functional example of a custom checked exception or comparable build on Task, for example, made some concepts more tangible than reading documentation alone. The first working version of other parts, particularly the file persistence code using `objectOutputStream` / `objectInputStream`, was accepted with only a cursory understanding of how java serialization walks the object graph, that section had to be written up separately in order to be truly understood rather than a whole response, AI aid enhances understanding the most.

4.3 When AI Should Not Be Used?

- When learning a basic idea for the first time is the aim of the exercise (for example, a first try at OOP inheritance), writing it by hand first develops comprehension that can then be compared to an AI's version.
- For security sensitive code (payment processing, cryptography, authentication) without professional human review, since minute errors can be hazardous and difficult to spot.
- AI generated APIs enter a project by accepting unverified code when the AI's recommendation cannot be independently checked (there is no means to compile, run or test it).
- In examinations, assessments or any other evaluation that is specially intended to gauge an individual's capacity without assistance.

4.4 Ethical Considerations in AI Generated Code.

- Honesty and attribution: As this report does in section 3.4, a report or submission should explicitly state which sections were AI- assisted.
- Licensing risk: Produced code can imitate licensed training data, for production programs, this needs to be verified rather than relied upon.
- Accountability : Any faults, security holes or improper behavior that appears in the finished product are developer's responsibility, not the AI tool's.
- Academic Integrity: Guidelines about the amounts of AI support allowed in coursework must be adhered to and disclosed as necessary.

5. Conclusion (Task 04)

The trade offs of AI Assisted programming become tangible rather than abstract when the same Task Management System was build twice. Version A demonstrates what a decent manual implementation would look like under typical time constraints, it functions, but it is brittle and can be crashed by a single incorrect user input. Enums, a custom exception type, and a few try/catch blocks transformed a program that crashes into one that degrades gracefully without altering the general design. Version B demonstrates how quickly that brittleness can be handled provided the AI is asked the correct questions.

The general conclusion drawn from this experiment is that AI is most useful in programming education and industry as a second set of eyes and an accelerant rather than as a substitute for comprehension. It excels at generating well known, structural code (validation, enums, boilerplates, exception types) that is simple to verify but time consuming to write, freeing you time to consider the system's actual architecture. The manual conventional version created initially for this project remained a helpful baseline for determining whether the AI Assisted version was indeed an upgrade rather than merely different because it is weakest and possibly dangerous precisely where independent verification is most difficult.

6. GitHub Repository

The repository below contains the whole source code, documentation and the commit history for both versions.

Repository: https://github.com/chalaniChamodya/Task_Management_System